

Briggs v4.7 — what changed, and why I think it works

A plain-language walk-through. No prior knowledge of the run assumed.

Couette line · 01_briggs/src/briggsv4.7.jl · written against the v4.6 run (contour_iteration_v4.6.json, 567 iterations) · 11 September 2026

The rule this code has to obey

v4.7 goes in **blind**. Nothing inside it knows where the pinch is — no ω_p , no α_p , no number computed from them. Section 9 is the audit, including a check I ran on the file itself. Where this document says the run was “too high” or “too far off”, that is *me* judging the run afterwards; it is never something the code was told. Almost every statement below is also written in a form the run could have made about itself, from its own output.

1 What the program is hunting, and the words for it

Start from the physics. For a given complex frequency ω , your Couette problem has a set of complex wavenumbers α that solve the dispersion relation. Numerically: fix ω , assemble the 300×300 generalised eigenvalue problem, and read off its spectrum of α .

Out of that spectrum the code follows two particular roots as ω moves around — one belonging to a wave carried one way, one the other. Call them the **upper branch** and the **lower branch**.

Following them just means: at each new ω , pick the root nearest to the one you had last time.

The **pinch point** is the one special ω where those two roots *collide*. That collision is what decides absolute versus convective instability, and finding it is the entire purpose of the run.

L — the sample line

A horizontal line of ω sample points. It has a **height** (its imaginary part, ω_{i}) and a set of **sample positions** along it (their real parts). “The grid” means those positions.

the gap (d_{branch})

At every sample on L you get one upper root and one lower root. The distance between them is $|\alpha^u - \alpha^l|$. The **gap** is the smallest such distance anywhere along L. If L passed exactly through the pinch, the gap would be zero. So the gap is the scoreboard: it is how the run knows whether it is winning.

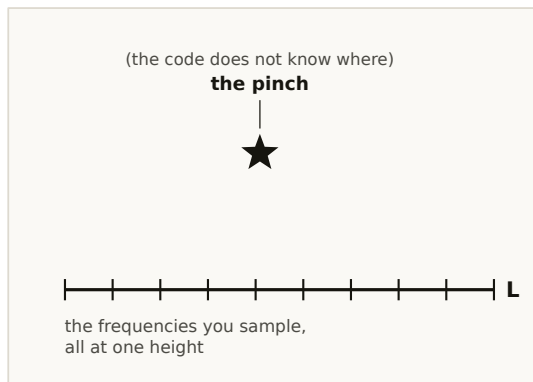
the miss

Your sample points are discrete. The pinch almost certainly sits *between* two of them. How far sideways the nearest sample sits from it is the **miss**. The run estimates this without knowing the answer, by fitting a parabola to the gap across a few neighbouring samples — section 7, Change 6.

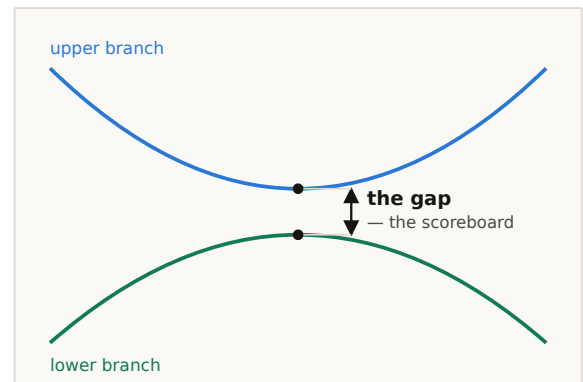
F

A second curve, this time drawn in the α -plane. F is not part of the scoring; F is what decides *how high to put L*. It turns out to be the thing that broke. Section 4.

the ω -plane — frequencies



the α -plane — wavenumbers



for each sample ω :
solve the 300×300
eigenproblem, keep
two of the roots

Figure 1. The two planes. Left: the line L of frequencies the run currently samples, and the pinch it is hunting — whose position it does not know. Right: for each sample it solves the eigenvalue problem and keeps two roots; as you walk along L, those two roots trace two curves that come close and then part again. The closest they get is **the gap**.

2 Why the gap is such a brutal scoreboard

When two roots of an equation merge, they merge like a square root. Concretely: if your frequency is off by a small amount ϵ , the two α roots come out separated by something proportional to $\sqrt{\epsilon}$, not by ϵ .

That one fact drives nearly everything that follows:

- a *tiny* error in ω shows up as a *large* gap. An error of 5×10^{-6} in the frequency is enough to hold the gap open at about 10^{-2} ;
- to **halve** the gap you must make ω **four times** better. Progress is expensive and gets more expensive;
- the gap tells you *that* something is off, never *which* thing.

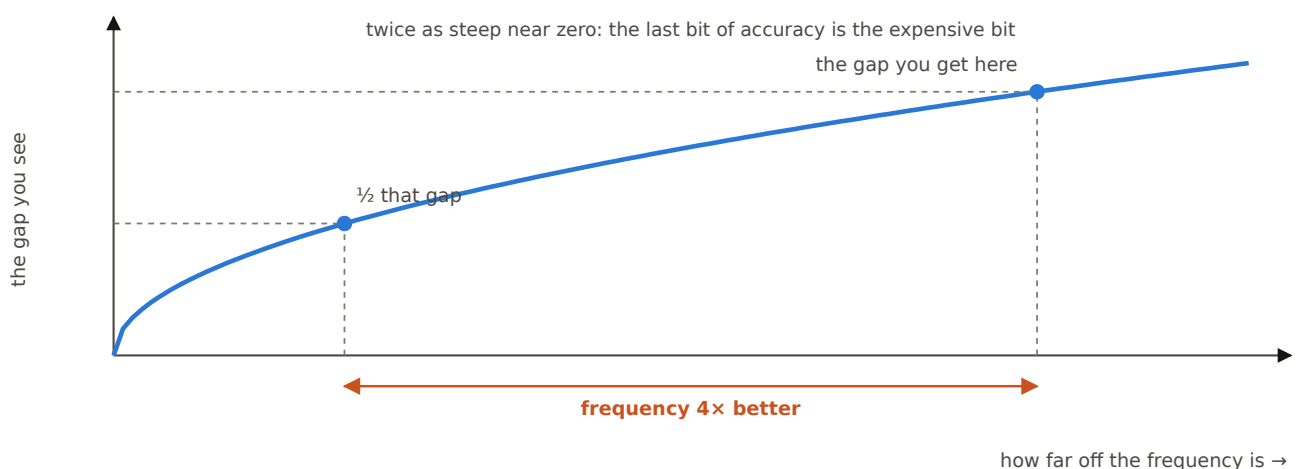
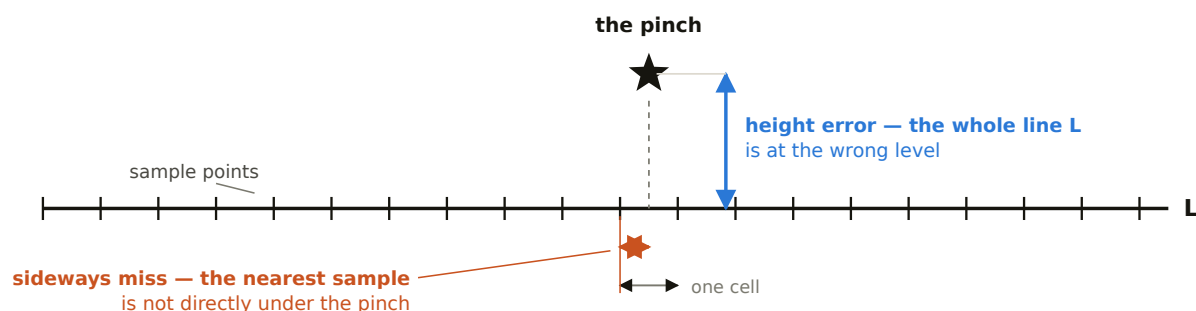


Figure 2. The square-root law. The gap grows steeply at first and then flattens, so near the answer a big improvement in the frequency buys only a small improvement in the score. Read it backwards and it is worse: four times better in ω , twice better in the gap.

There are exactly two ways the frequency can be off, and the gap responds to whichever is bigger.



the gap is the SQUARE ROOT of the distance from the nearest sample to the star — so both arrows get magnified

Figure 3. The two error components. **Sideways:** the nearest sample is not directly under the pinch. **Height:** the whole line L is at the wrong level. v4.6's job was the sideways one, and it did it — which is exactly how the height one ended up in charge.

3 Where v4.6 got to, and where it stopped

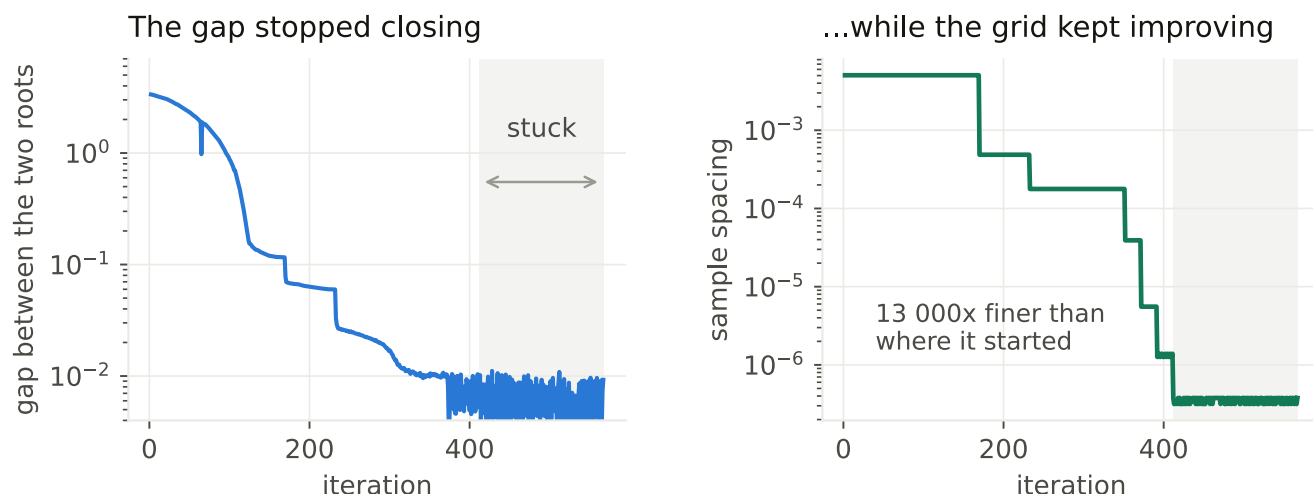


Figure 4. The v4.6 run, from its own stored output. Left: the gap fell by three orders of magnitude and then sat at roughly $7\text{--}9 \times 10^{-3}$ for its last 150 iterations (shaded). Right: over that same stretch the sample spacing was still improving. The thing that was supposed to be the bottleneck was being fixed, and the score stopped responding.

Three facts, all of which the run could state about itself without knowing the answer:

1. **The grid kept getting better.** Sample spacing down to 3.8×10^{-7} , and the run's own parabola fit put the remaining sideways miss at around 3×10^{-7} . That is three to four times finer than it needed to be to support the gap it was actually reporting.
2. **The gap stopped falling anyway**, at about 7×10^{-3} .
3. **The height went back up.** Between iteration 400 and iteration 567, the run's own ω_i rose by 3.7×10^{-6} — it had been coming down.

Fact 3 is the giveaway, and it needs no reference to the true answer at all. Nothing about the grid can push the height up. Something else moved it. To see what, you have to look at F.

4 What actually sets the height: F, and a mountain pass

F is a curve in the α -plane. Every point on it has a frequency attached, so F has an image in the ω -plane, and that image has a highest point. **The code parks L just above that highest point** — 10^{-9} above it, via the bisection that v4.4 and v4.6 built.

Why that is the right rule is easiest to see as terrain. Think of the α -plane as a map, and of the height of the frequency at each point as elevation. The pinch is a **mountain pass**: a saddle. Walk over the pass along the correct route and the highest ground you touch *is* the pass. Get shoved sideways and you cross the ridge higher up, and you report a summit that is too high.

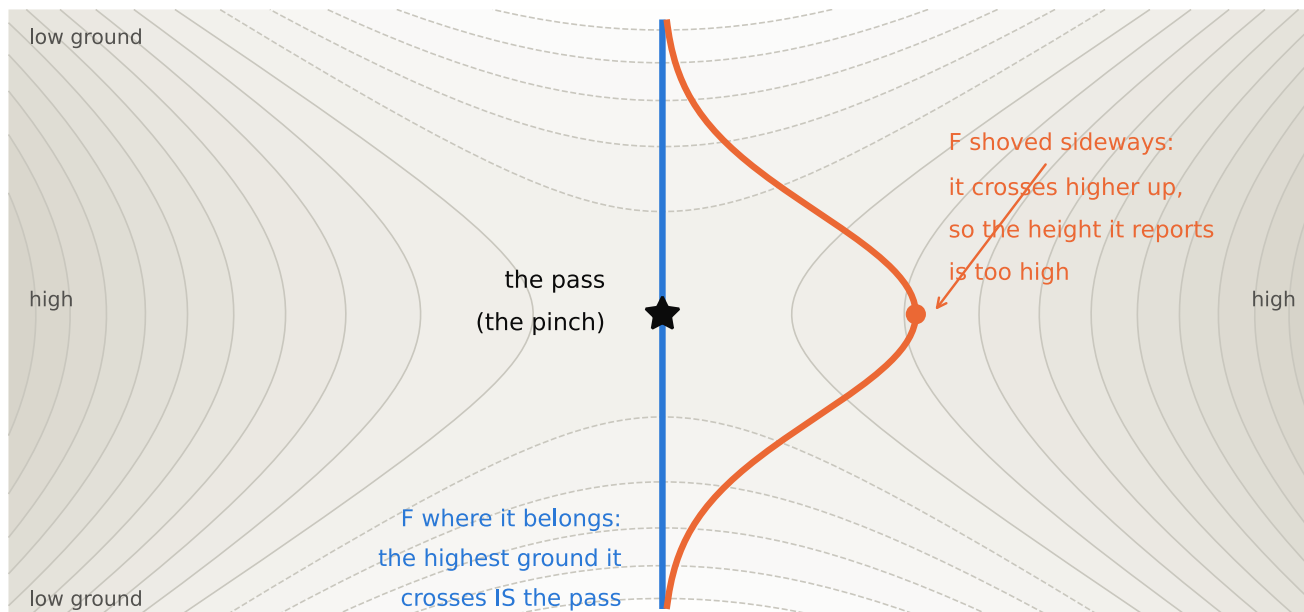


Figure 5. The pinch as a mountain pass. The rule “put L just above the highest point F reaches” is exact when F crosses at the pass, and reports too high a value for every other route. This is the link between a badly placed F and a height error — and, through the square-root law, a gap that will not close.

So the causal chain is: **F → the height → the gap**. Now, what moves F?

F is evolved by a simple flow. Each point of it feels two things: a **push away from both branch curves**, and a smoothing term that stops it kinking. The push comes from a potential

$$\phi = \exp(\zeta / r^2) - 1 \quad r = \text{distance from that point of F to a branch point}$$

ζ (“zeta”) is a single number that sets how far the push reaches. **$\sqrt{\zeta}$ is its range**: much further away than $\sqrt{\zeta}$, the exponent is tiny and there is effectively no push; at about $\sqrt{\zeta}$ the push is order one; much closer than $\sqrt{\zeta}$, it explodes.

The push exists for a concrete reason. If F ever lands exactly on a branch point, the eigen-solve blows up — that was the crash in v4.3a. ζ is a safety rail.

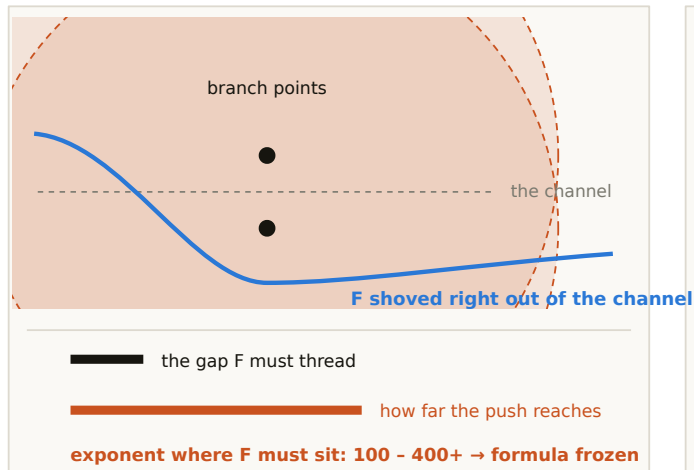
5 The bug: a safety rail that grew into a wall

ζ was a hand-tuned constant, 4×10^{-4} . So the rail reached $\sqrt{\zeta} = 0.02$, and it reached that far *forever*, no matter what else changed.

But the channel F has to thread is the gap — and the gap shrinks as the run succeeds. That is the trap:

- at a gap of 3.1×10^{-2} , a rail of 0.02 is about two thirds of the channel width. A rail;
- at a gap of 5×10^{-3} , the same 0.02 is **four times the entire channel**. The two rails overlap across the whole of it, and there is nowhere left to stand.

v4.6 — the push reaches a fixed 0.02



v4.7 — the push reaches 65% of the gap

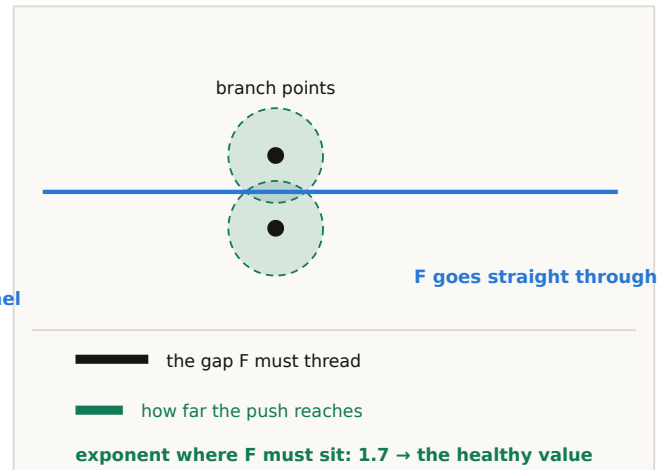


Figure 6. The squeeze. Same two branch points, same channel, two different rules for how far the push reaches. On the left the push covers the entire neighbourhood and F is shoved out of the channel altogether; on the right the rail is tied to the channel and F passes through it, feeling exactly the push it felt back when this worked.

And it is worse than merely “too strong”. Where F has to sit, roughly half the gap from each branch point, the exponent in that formula is about $4\zeta/d^2$. With ζ frozen, that grows like $1/d^2$. The code clips the exponent at 400 to avoid an overflow, and the resulting force is clipped again at ± 100 . Past the ceiling **the push no longer depends on distance at all**: it becomes a constant, maximum-strength shove applied over a whole region. F stops being guided and starts being kicked.

What the v4.6 output actually shows (recomputed here from its stored contours, not taken on trust)

- over the last 156 iterations the largest exponent had a **median of 111**, and went past the 400 ceiling on **26 of them** (31 frames across the whole run, all of them after iteration 392);
- F's closest approach to a branch point fell to 1.2×10^{-4} — deep inside the rail, not beside it;
- the frame-to-frame jitter in the gap was about 2.8×10^{-3} , against 3.6×10^{-4} predicted by the round-off model fitted during the v4.5 analysis. That is **$7.6 \times$ too large to be arithmetic**.

Figure 9, on page 8, plots that exponent over the whole run.

The last one is the one that matters most, because it is independent of the rest of the story. If v4.6 were sitting on the double-precision floor of this problem, its jitter would *be* the size the round-off model predicts. It is not. Something mechanical is kicking the contour, and there is real room left underneath.

6 Change 5 — tie the rail to the channel (the real fix)

Instead of holding ζ fixed, hold fixed the thing that was *measured healthy*:

$$\zeta(d) = 4 \times 10^{-4} \times (d / 3.08 \times 10^{-2})^2$$

which keeps two quantities constant at every gap size, forever:

what stays fixed	value	why that value
the exponent where F has to sit	1.687	it was ~ 1.7 on the v4.4/v4.5 plateau, where the run was behaving
the push range as a fraction of the channel	65 %	same plateau: F threaded the channel cleanly, ratio 1.00

The calibration point — $\zeta = 4 \times 10^{-4}$ at a gap of 3.08×10^{-2} — comes from this project's own run history, not from anywhere near the answer. ζ is **capped** at the old hand-tuned value, so it can only ever shrink; formula and cap meet exactly at the calibration point, so nothing jumps at the handover.

Two guards, because this closes a loop

ζ now depends on the gap, the gap depends on the height, the height depends on F, and F depends on ζ . That is a feedback loop, so it gets damping:

- the input is a **running median of the gap over 25 iterations**, not the raw gap. The raw value swings has a standard deviation of about a third of its median at this level, and $\zeta \sim d^2$ would roughly double that into a $\pm 65\%$ kick every iteration;
- ζ may change by at most **3 % per iteration**, so the landscape never jumps out from under F. Going from 4×10^{-4} down to the 4×10^{-7} a gap of 10^{-3} would ask for takes about 230 iterations.

It is a no-op everywhere v4.6 was fine

The median window has to fill before ζ moves at all, and ζ is capped at the old value. Replaying the rule against v4.6's own recorded gap history, **ζ first leaves 4×10^{-4} at iteration 246** — so the whole descent, the part that worked, runs at exactly the v4.6 setting. Setting `ZETA_ADAPTIVE = false` restores v4.6 exactly.

7 The other two changes — removing blockers

Change 6 — stop believing an impossible number

`grid_share` is the run's own estimate of *what fraction of the gap I see is caused by my sideways miss*. It decides whether refining the grid is worth doing: 0.90 or more means “yes, the grid is what's holding me back”. It is computed by fitting a parabola to the gap across five neighbouring samples.

When the *height* error dominates, that fit degenerates: the part of the curve that varies across five samples is a couple of percent of the whole, so the fitted curvature is noise and collapses — and `grid_share` comes out at 200, 400, up to 580. A test that reads “ ≥ 0.90 ” sails straight past that and calls it a strong signal. It happened on **48 of 148** scored frames in the last stretch of v4.6.

what fraction of the gap does the sideways miss explain?

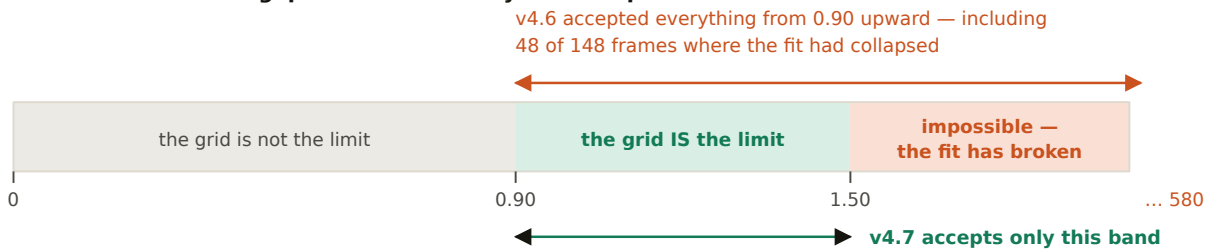
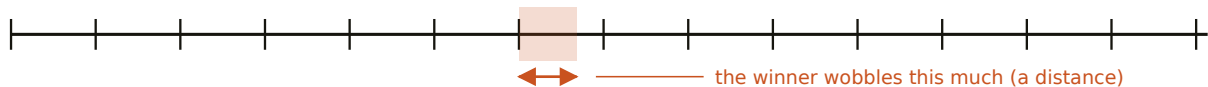


Figure 7. One-sided versus two-sided. A share far above 1 is not a stronger signal, it is a broken fit — the run telling you the grid has stopped being the limiter. v4.7 accepts only the band 0.90–1.50. This cost nothing on v4.6 (more refinement would not have helped there anyway), but it was wrong.

Change 7 — ask “has it stopped moving?” in a unit that doesn't change

Refinement only fires when the run decides it has stalled, and part of that test was: *has the winning sample stayed within ± 1 index for 20 iterations?* On a uniform grid, ± 1 index is ± 1 cell — a distance. v4.6 does not have a uniform grid: vertex placement drops a cluster of a dozen nodes inside one old cell, so the winner hops 6 to 9 indices while barely moving at all. The test could never pass again, and **v4.6 froze at refinement level 6 for its last 150 iterations.**

before refinement — ticks evenly spaced



it spans less than one tick, so the stall test passes and refinement fires

after refinement — a dozen ticks packed into one old cell



the same distance now spans 8 ticks — the test can never pass again, and v4.6 froze

v4.7 asks the same question in ω , against the local tick spacing — a unit refinement cannot corrupt

Figure 8. Counting ticks stopped being the same as measuring distance. v4.7 measures the spread of the winning frequency against the local cell width instead (± 2 cells). Same question, asked in a unit the refinement does not corrupt — and on a uniform grid it reduces to exactly the old test.

8 Why I think it works — and how it could be wrong

1. **The diagnosis does not depend on the fix being right.** The jitter is $7.6\times$ larger than round-off can account for. That single measurement says the run is not at the numerical floor, whatever the cause.
2. **The mechanism was found, not guessed.** The run's own stored contours show F inside the rail, with the exponent past the ceiling on 26 of the last 156 frames and a median of 111. That is output, not inference.
3. **The fix restores a state that was measured working** rather than introducing a new tuned constant. The two numbers it pins — exponent 1.687, range 65 % of the channel — are the values

the healthy plateau had.

4. **It cannot disturb what already worked**, being capped at the old value and inactive until iteration ~ 246 .
5. **The replay is strongly in its favour**. Applying the v4.7 rule to v4.6's own recorded gaps and its own recorded contour geometry, over the 322 frames from iteration 246 on (where ζ first moves): frames with an exponent above 100 fall from **136 to 16**, and frames past the 400 ceiling from **31 to 2**. Taking just the last 156 frames, the median largest exponent falls from **111 to 5.5**.
6. **The other two changes only remove blockers**. One lets the refinement test fire again; the other stops a degenerate fit from reading as confidence. Neither adds new behaviour.

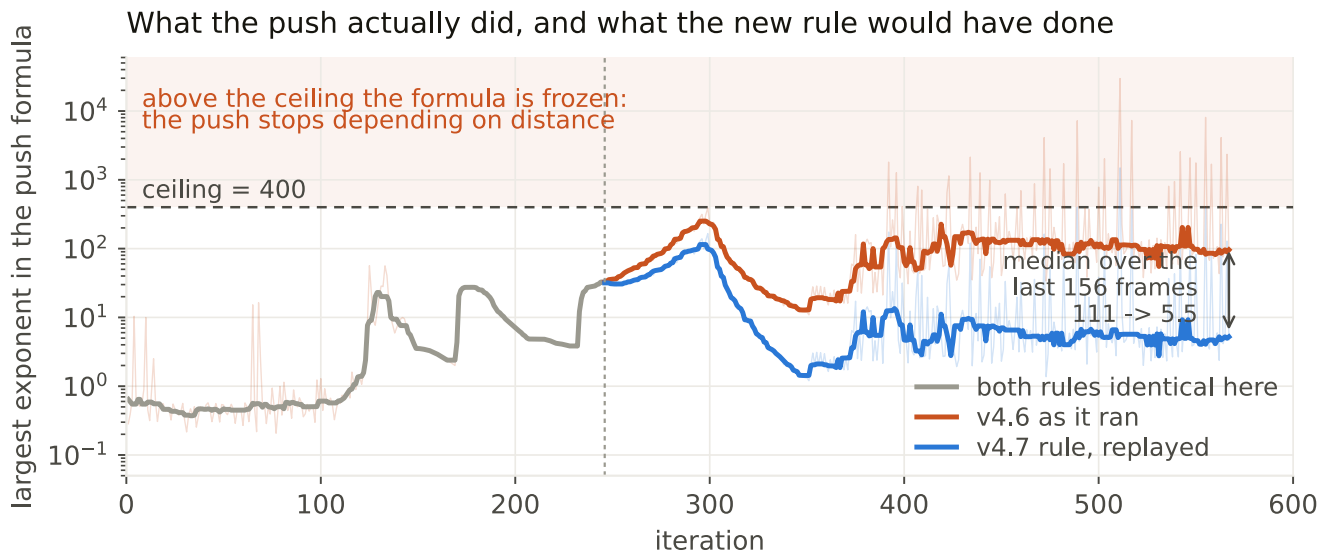


Figure 9. The replay, drawn from v4.6's own output. Grey: the stretch where the two rules are by construction identical, because ζ has not moved yet. Orange: what v4.6 actually did — drifting up into the hundreds and repeatedly across the ceiling. Blue: what the v4.7 rule would have asked for on the same frames. Thick lines are 9-frame medians, faint lines the raw values. What is plotted is the largest exponent anywhere on F — which is exactly the new $xarg=$ column, so you can compare the live run against this directly. Indicative rather than predictive: the exponent is evaluated on v4.6's contour geometry, and F will sit differently once ζ changes — which, since that geometry is the damaged one, makes it pessimistic.

What success should look like in the console

$xarg=$ (the largest exponent this iteration) sitting in single digits instead of the hundreds. $z=$ (ζ) drifting smoothly downward, not oscillating. The height coming back down instead of drifting up. The gap falling from $\sim 7 \times 10^{-3}$ toward $\sim 2 \times 10^{-3}$, which is what the grid already supports, and then toward $\sim 1.6 \times 10^{-3}$, which is where the round-off model says signal equals noise for this 300×300 problem.

That last number is a real wall, not a tuning failure.

Where I could be wrong

- **The replay is not clean.** Two of its 322 frames still cross the ceiling (iterations 489 and 511), because on those frames v4.6's F had already been shoved to within 1.2×10^{-4} of a branch point. The replay inherits v4.6's bad geometry — the very thing the fix is meant to prevent — so it is pessimistic, but it is not a proof.
- **ζ is now a feedback loop.** The median and the 3%-per-iteration limiter are damping, not a stability proof. A slow oscillation in the $z=$ column would be the symptom.

- **The rail is genuinely weaker in absolute terms.** It exists because of a real crash. v4.7 keeps the protection constant *relative to the channel*, which I believe is the right invariant — but it has never been run below a gap of about 7×10^{-3} .
- **The falsification test is clean.** If xarg stays under ~ 10 and the gap still refuses to fall, the diagnosis is wrong. Next two suspects, both already documented: the second-derivative formula used on F assumes equal spacing on both sides and does not even vanish for a straight line on a non-uniform grid; and the 15-point rolling average would smear a refined cluster against its coarse neighbours.
- **This run will not stop on success.** PINCH_TOL is set to 5×10^{-4} , deliberately below anything reachable, so you will be reading a plateau off the curve rather than getting a “converged” flag. ADAPT_D_FLOOR = 2×10^{-3} stops the refinement, not the run.

9 The blind audit

What v4.7 reads while it runs: the gap it just measured; the profile of gaps along L; its own sample positions; its own F, and the frequencies of the points on it; its own distances to the two branches. Every one of those is produced by the run itself.

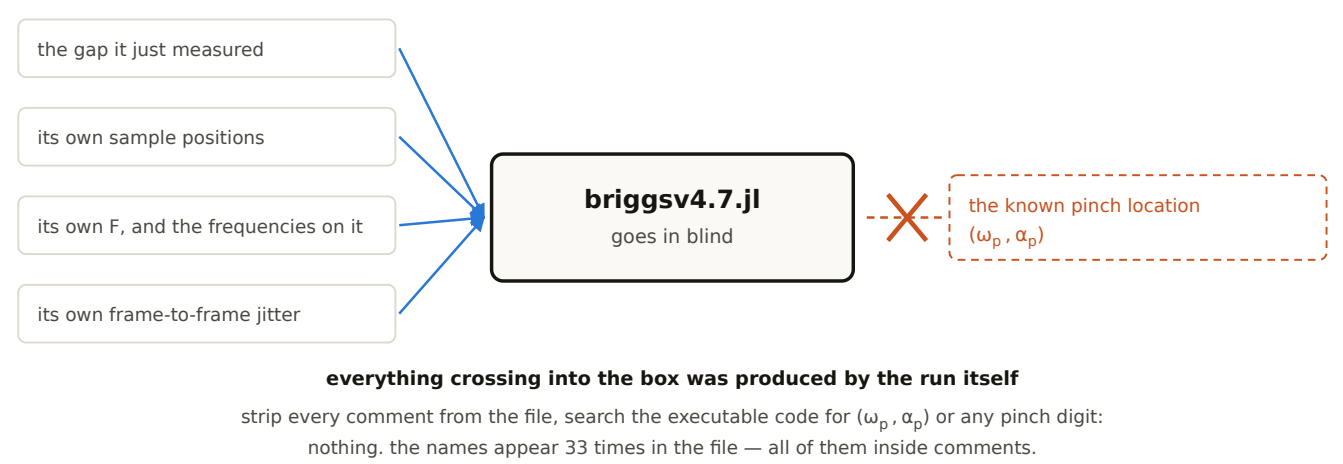


Figure 10. Everything crossing into the code is something the run produced. I checked rather than assumed: stripping every comment from briggs4.7.jl and searching the executable code for ω_p, α_p or any of the known pinch digits returns nothing at all. Those names appear 33 times in the file — all of them inside comments that justify a choice.

constant	value	where it comes from
ZETA_REF / ZETA_D_REF	4e-4 at 3.08e-2	a working setting from this project's own earlier runs
the d ² law itself	—	from the requirement that F fit between two branch points — not from where those points are
grid-share bounds, window sizes, rate limit	0.90–1.50, 25, 3 %	thresholds on ratios of quantities the run measures about itself
ADAPT_D_FLOOR, PINCH_TOL	2e-3, 5e-4	derived from K, which was fitted from the run's own frame-to-frame jitter

One honest exception, carried over from v4.6

ADAPT_FIT_HALF = 2 — the width of the stencil used for the parabola fit — was chosen by comparing candidate widths *against the analytic pinch*. The code never reads that value, but my choice of the constant was informed by the answer. The justification that does not need the answer is the ordinary one: a narrower stencil has less truncation bias, and 2 is the smallest that over-determines a parabola. The fully clean version is to pick the stencil at run time from the agreement between the n=2 and n=3 fits, which needs no external reference at all. Not implemented.

10 The three changes at a glance

change	in one line	switch it off with	if I have this wrong
5 — ζ tied to the gap	the push that keeps F off the branch points now shrinks as the gap closes, so its reach stays 65 % of the gap instead of growing to 4× it	ZETA_ADAPTIVE = false	nothing is lost above a gap of 3.08×10^{-2} , where ζ is capped at the old value. Below it the rail is weaker in absolute terms, so the thing to watch for is F touching a branch point — the guard added in v4.3a still catches the direct failure
6 — grid_share bounded above	a share far above 1 now reads as a collapsed fit rather than a strong signal	ADAPT_GRID_SHARE_MAX = Inf	little to lose: it only stops the run from scoring a round on a meaningless number
7 — stall measured in ω	“has the winner parked?” is asked in frequency against the local cell width, not in grid-index counts	ADAPT_STALL_BY_OMEGA = false	the run may now refine more often than v4.6 did; the point budget, the level cap and ADAPT_D_FLOOR still bound it

Carried over from v4.6 unchanged: ADAPT_SCORE_GATE, ADAPT_VERTEX_PLACE, OMEGA_LB_PARABOLIC and OMEGA_BISECT_ENGAGE = 1e-2. New per-iteration fields in the JSON: zeta_alpha, zeta_d_med, exp_arg_max, dist_u, dist_l. New console columns: z= (now the live ζ , where it used to print the frozen one) and xarg= (the largest exponent this iteration — the quantity plotted in Figure 9).

Numbers in this document were recomputed from contour_iteration_v4.6.json (567 iterations) and read from briggs_v4.7.jl directly, rather than copied from the design notes. The v4.7 replay applies the new ζ rule to v4.6's recorded gap history and evaluates the exponent on v4.6's recorded contour geometry; it is indicative, since F will sit differently once ζ changes.